

Lecture 5

Software development life cycle

Outline

- Project life cycle
- Agile – scrum
- SQA components in project life cycle

References

- Daniel Galin, Software Quality Assurance: From theory to implementation, chapter 7.1
- Ken Schwaber & Jeff Sutherland, The Scrum Guide: The Definitive Guide to Scrum: The Rules of the Game

Project life cycle

Development Models

General Models:

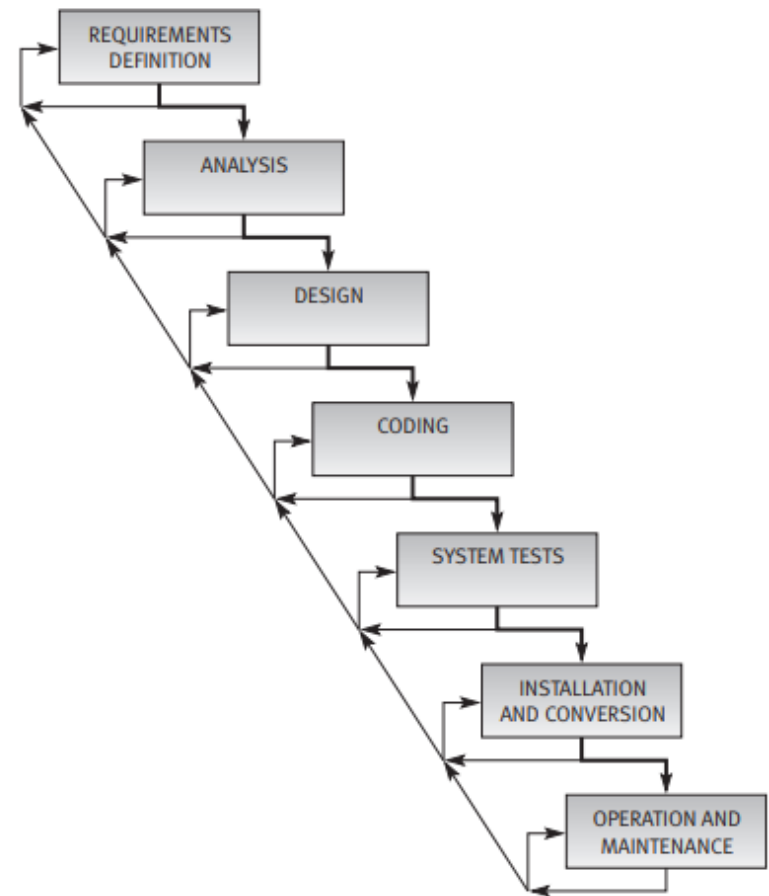
- **The Waterfall Model**
- **The Prototyping Model**
- **The Spiral Model**

Note:

- They refer to different development contexts
- Each model is suitable for different project contexts

The Waterfall Model

- The **classic model**
- Provides the most comprehensive description of the process available
- Displays major building blocks as a **linear sequence**



Seven phases of Waterfall model

The Waterfall Model

1. Requirements Definition:

- Customer defines functionality
- Establish cooperation and component interfaces

2. Analysis:

- Analyze requirements' implications
- Form the initial software system model

3. Design:

- Detailed definition of inputs, outputs, and processing procedures
- Data structures, databases, and software structure

The Waterfall Model

4. Coding:

- Translation of design into code
- Involves quality assurance activities: inspection, unit tests, and integration tests

5. System Tests:

- Uncover as many errors as possible to achieve acceptable quality
- Performed by the developer before supply
- Acceptance tests are usually conducted by the customer or an independent party.

The Waterfall Model

6. Installation and Conversion:

- System is installed to serve as firmware
- Initiation of a software conversion process (if replacing an existing system) to ensure uninterrupted activity

7. Regular Operation and Maintenance:

- The longest period (usually lasts for several years)
- Three types of maintenance services:
 - *Corrective*: Repairing software faults
 - *Adaptive*: Fulfilling new requirements
 - *Perfective*: Adding minor features to improve performance

The Waterfall Model Review and Evaluation

End of Phase Review:

- Outputs are examined by the developer and often by the customer

Possible Outcomes:

- **Approval** of outputs and progress to the next phase
- **Correction/Change** demands for parts of the last phase
- **Return** to earlier phases (required in certain cases)

The Prototyping Model

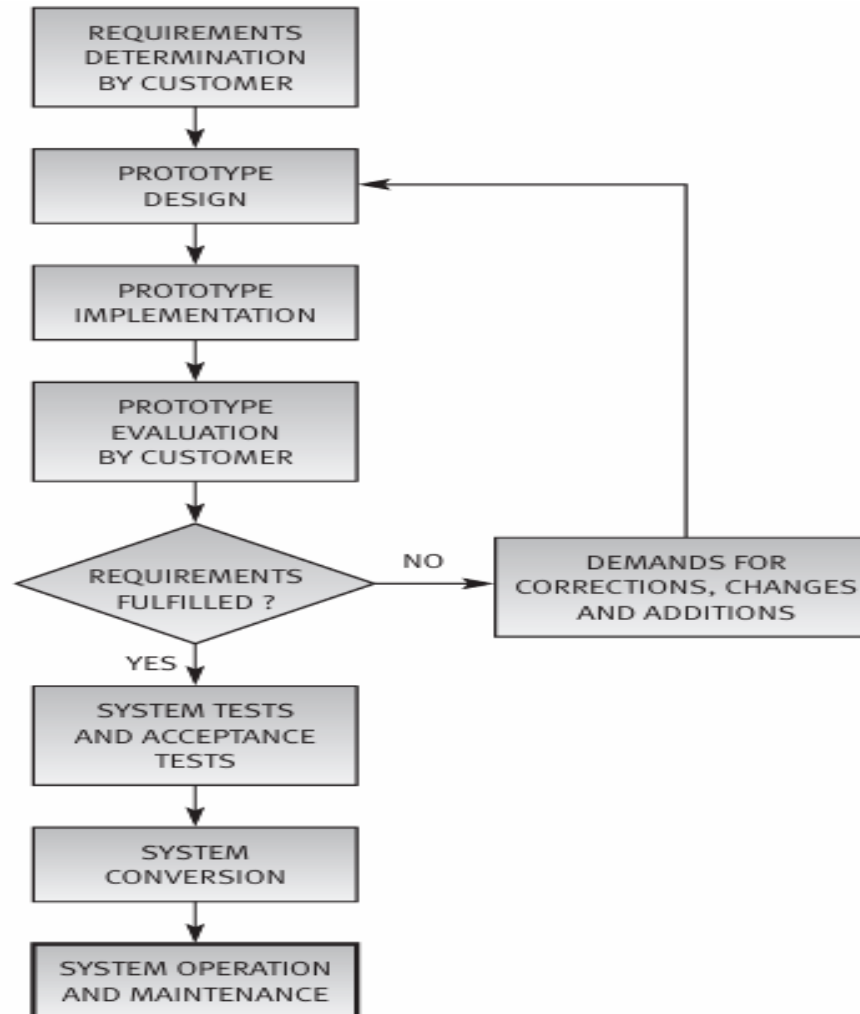
Methodology Basis:

- Uses advanced application generators (fast development)
- Requires active participation by customers/users capable of examining prototypes

Evolutionary Process:

- Repeat process till the prototyping goal is achieved
- Software prototypes are used for **communication** between the developer and the users

The Prototyping Process



Scope and Pros/Cons

Scope:

- Efficient and effective mainly for **small- to medium-sized** projects.

Advantages:

- Shorter development process
- Substantial savings of development resources (man-days)
- Better fit to customer requirements and reduced risk of project failure
- Easier and faster user comprehension of the new system

Deficiencies:

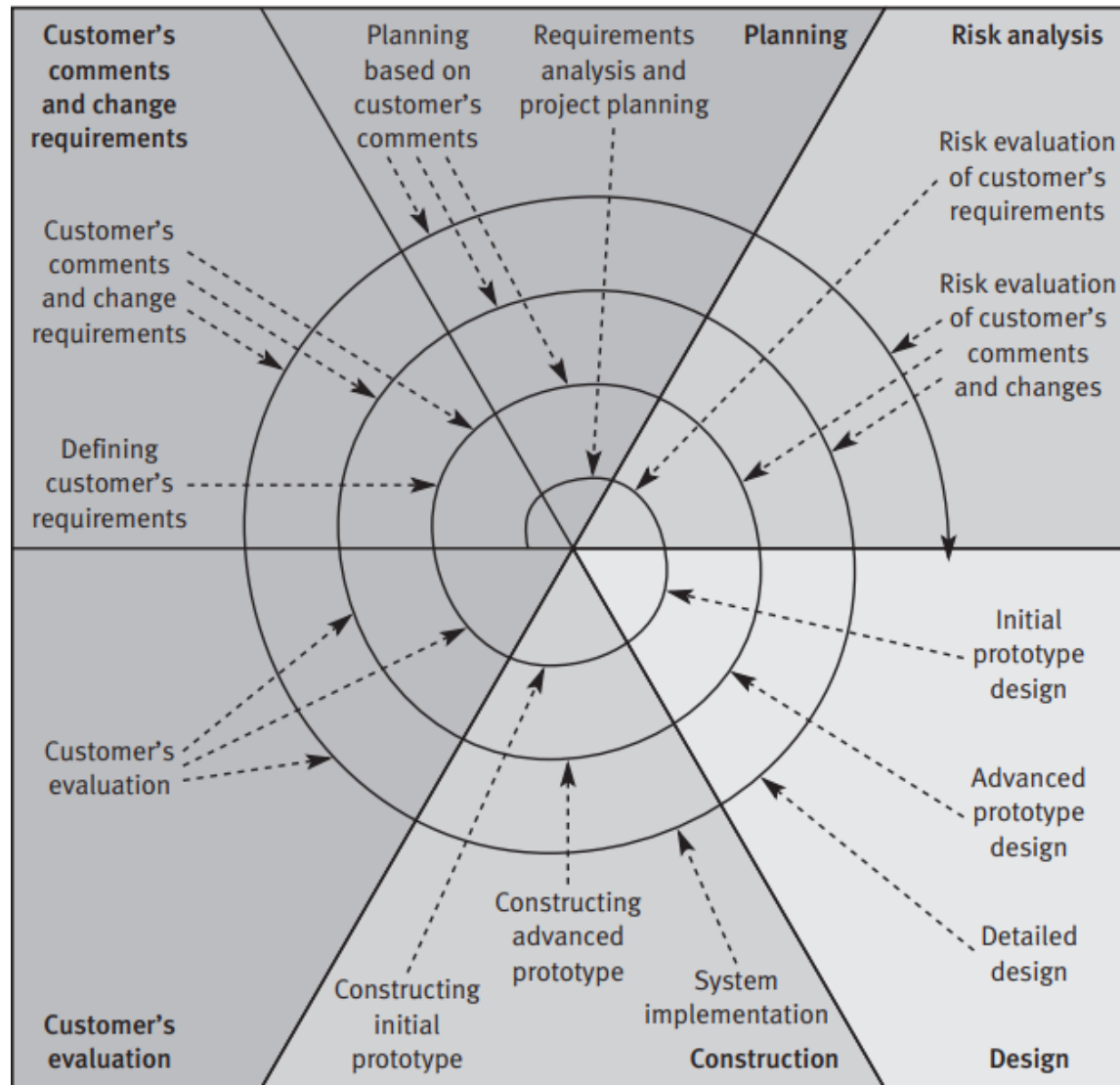
- May neglects software quality, long-term maintainability

The Spiral Model

The spiral model: the combination of

- the iterative nature of prototyping
- the controlled and systematic aspects of the waterfall model.

The Spiral Model



The advanced spiral model (Boehm, 1998)

The Spiral Model Pros/Cons

Pros:

- Can apply the prototyping approach at any stage
- Can change requirements in later phase
- Suitable for large projects

Cons:

- Difficulty in time management.
- Easy to be a failure without consideration of risks at each iteration
- Time-consuming

Agile – scrum

What is Agile?

Agile Manifesto:

We have come to value:

- Individuals and interactions over processes and tools
- Working software over comprehensive documentation
- Customer collaboration over contract negotiation
- Responding to change over following a plan

That is, while there is value in the items on the right, we value the items on the left more

Agile is a set of **values** and **principles** that guide and shape development.

Agile methods Model

Model that approaches to Agile methods:

- Extreme Programming (XP)
- Scrum
- Kanban
- Crystal Agile Framework
- Dynamic System Development Method (DSDM)
- Feature-Driven Development (FDD)

Scrum

- Scrum is a lightweight framework that helps people, teams and organizations generate value through adaptive solutions for complex problems
- It is purposefully incomplete, only defining the parts required to implement Scrum theory
- Scrum Theory (**Empiricism** and **Lean Thinking**):
 - **Transparency:** The emergent process and work must be visible to those performing and receiving the work
 - **Inspection:** Frequently inspecting artifacts and progress to detect undesirable variances
 - **Adaptation:** Adjusting the process or materials as soon as possible if there is a deviation

Scrum Values

Successful use of Scrum depends on people becoming proficient in living these five values:

1. Commitment: To achieve goals and support each other
2. Focus: On the work of the Sprint to make the best possible progress
3. Openness: About the work and the challenges
4. Respect: For each other as capable, independent people
5. Courage: To do the right thing and work on tough problems

The Scrum Team

Structure:

- The fundamental unit is a small team consisting of one Scrum Master, one Product Owner, and Developers
- Typically, 10 or fewer people
- There are no sub-teams or hierarchies

Key Characteristics:

- **Cross-functional:** Members have all the skills necessary to create value each Sprint
- **Self-managing:** They internally decide who does what, when, and how
- The entire team is accountable for creating a valuable, useful Increment every Sprint

Accountabilities: Product Owner & Scrum Master

Product Owner (PO):

- Accountable for **maximizing the value** of the product
- Manage the **Product Backlog**: Developing the Product Goal, ordering items, and ensuring transparency
- The PO is one person, not a committee

Scrum Master (SM):

- Accountable for establishing Scrum as defined in the Scrum Guide
- True leaders **who serve** the Scrum Team and the organization
- Accountable for the team's effectiveness and removing impediments

Accountabilities: Developers

Role:

- The people in the Scrum Team that are committed to creating any aspect of a usable Increment each Sprint

Key Accountabilities:

- Creating a plan for the Sprint (**Sprint Backlog**)
- Instilling quality by adhering to a **Definition of Done**
- Adapting their plan each day toward the **Sprint Goal**
- Holding each other accountable as professionals

Scrum Events

The Sprint:

- The heartbeat of Scrum, where ideas are turned into value
- Fixed length of one month or less
- During the Sprint:
 - No changes are made that would endanger the Sprint Goal
 - Quality does not decrease
 - The Product Backlog is refined as needed
 - Scope may be clarified and renegotiated with the PO as more is learned

Sprint Planning:

- Initiates the Sprint by laying out the work to be performed
- Addresses three topics: Why is this Sprint valuable? What can be Done? How will the chosen work get done?

Scrum Events

Daily Scrum:

- A 15-minute event for Developers to inspect progress toward the Sprint Goal
- Produce an actionable plan for the next day of work

Sprint Review:

- Inspect the outcome of the Sprint and determine future adaptations
- The team presents the results to key stakeholders

Sprint Retrospective:

- Plan ways to increase quality and effectiveness
- Concludes the Sprint

Scrum Artifacts

Designed to maximize transparency of key information

- **Product Backlog:**
 - An emergent, ordered list of what is needed to improve the product
 - The single source of work undertaken by the Scrum Team
- **Sprint Backlog:**
 - Composed of the Sprint Goal (why), selected items (what), and the plan (how)
- **Increment:**
 - A concrete stepping stone toward the Product Goal
 - Must be usable in order to provide value

The Commitments

Each artifact contains a commitment to ensure transparency and focus:

- **Product Backlog: Product Goal**
 - Target future state for planning
- **Sprint Backlog: Sprint Goal**
 - Single objective ensuring coherence
- **Increment: Definition of Done**
 - Formal quality standard
 - No Definition of Done = No release

SQA components in project life cycle

SQA components in project life cycle

Project life cycle encompasses two stages:

- Development life cycle stage
- Operation–maintenance stage

Development life cycle SQA components:

- Goal: To detect design and programming errors during the design and coding phases
- Four sub-classes:
 - Formal design reviews
 - Peer reviews
 - Expert opinions
 - Software testing

SQA components in project life cycle

Operation–Maintenance Stage SQA Components:

- **Corrective:** Uses specialized SQA components
- **Improvements:** Reuses development components

External Participants (Subcontractors):

- **Goal:** Assure quality of outsourced parts
- **Purpose:** Reduce high risks (functional failures & delays)

Implementation Strategy:

- **Selection:** Determined by project characteristics
- **Timing:** Must be decided **before** project initiation

Summary

- Project life cycle
- Agile – scrum
- SQA components in project life cycle